DocQuorum: adaptive reading, quorum selection and focused re-reading

All steps run automatically • Python controls the workflow; models read the evidence

1 Inputs

PDF / images / native files

PDF XLSX

XLSX, DOCX, HTML, CSV, text

JSON Schema / template

{ }

Optional task (added to schema)

2 Prepare & route CODE

Normalize to PDF
Native text preserved; images oriented, flattened and small images enlarged

Inspect pages, text and layout
Pillow / native helpers • Poppler

MinerU for scans, MODEL **if available**
Cached text blocks + HTML tables

Layout first; otherwise text + page count

3 Read MODEL

A Specialized layout
Column reader OR ruled-table reader → target list
Reader agent → remaining fields
Combine disjiont fields → L
Column crops · Ruled bands
No records / unreliable rules → B-D routing

B Text available / native file
Primary reader P
Text + page images when eligible

C No text • ≤12 pages
Primary P MODEL · Parse M* MODEL · One-shot O MODEL
Separate runs → up to 3 candidates

D No text • >12 pages
Primary P MODEL · Parse M* MODEL
Separate runs → up to 2 candidates

P / M: same adaptive agent, different evidence. **O:** one-shot call.

4 Check independently CODE

L: output checks

P: repairs → output checks

P: repairs → output
M: spacing → repairs → output
O: output checks

P: repairs → output
M: spacing → repairs → output

Schema + cource evidence; no model call.
Shape / biank values • rom conistency • printed values

5 Select CODE

Keep L

Keep P

C and D candidates
ONE candidate → keep it
MULTIPLE → QUORUM
Compare pairs; highest mean agreement
Keep one whole reading

Selected JSON

Prefer non-looped readings when available.

6 Zoom re-read MODEL CODE

Zoom eligible? — No → Keep selected JSON

Quorum result + zoom enabled + parser available

Yes

Disputed fields only
- MinerU locates source block
- Read sharp crop (300 dpi)
- Blind read: no candidate answers shown
- Guarded correction or keep value

7 Return

Keep selected JSON

{ }

Final JSON + report

Restore template shape

Inside P / M: bounded agentic reader

Python manages state and tools; the extraction model supplies values

1 PLAN CODE
Choose eligible scope
- whole document
- schema groups
- text windows
- scalars + list chunks

2 READ MODEL
Schema-guided extraction
Text and/or page images when eligible

3 OBSERVE CODE
Cut-off?
Missing keys?
Repetition loop?
Empty despite rows?

Retry / fallback needed? — No → Candidate JSON

Yes → **4 ADAPT** CODE
Larger budget; nullable required keys; split scope; repetition penalty

Observed result changes next call.

Scopes: whole document / schema groups / text windows / scalars + list chunks
For list chunks: copy opening-page legends + entity facts; attach relevant context to each chunk

Inside zoom: a source read with guarded updates

1 Disputed fields (up to 40)
Fields with low agreement or conflicting values

2 MinerU block → 300-dpi crop
Locate source block and read a sharp crop (300 dpi)

3 Independent model read
Field description + crop; no candidate answers

4 CODE **UPDATE only when both guards pass:**
- Re-read matches an existing proposed value
- Its original support + 2 zoom votes beats the kept value's support

Both pass → apply that proposal
Otherwise → keep selected value

Zoom never introduces a new value.

CODE = code (Python control) · MODEL = learned reading/parsing · = adaptive P/M loop

* Parse M requires cached evidence. Images depend on reading scope; OCR images take precedence when supplied.
Successful parsin ṡnorm. Zoom is enabled by default after multi-reader quorum; parser required, -no-zoom disables it. | No answer keys. Task text extends the schema description. |